

10-Day Principal Engineer Interview Plan

Google • Adobe • D.E. Shaw | Target: Senior/Principal Systems Engineering

Primary positioning: C++ • Distributed Systems • System Design • Cloud • AI • Performance. Your Microsoft/WinUI experience remains valuable; position it as part of a broader systems-engineering career.

Day 1 — Baseline + C++ Core

- C++: RAI, pointers/references, const correctness, copy/move semantics, smart pointers, virtual functions/vtables.
- DSA: arrays, strings, hash maps, two pointers, sliding window.
- Solve 5–6 medium problems under time limits.
- Create a one-page C++ cheat sheet from memory.
- Write your 90-second introduction and practice it 3 times.

Day 2 — Modern C++ + Concurrency

- C++: STL, lambdas, templates, move semantics, C++17/20 basics.
- Concurrency: threads, mutex, condition_variable, atomics, race conditions, deadlocks.
- DSA: binary search, stacks/queues, heaps.
- Practice explaining time/space complexity aloud.
- Do one 45-minute timed coding session.

Day 3 — Trees, Graphs + C++ Systems

- DSA: trees, BFS/DFS, graphs, topological sort, intervals.
- C++: memory layout, allocation, object lifetime, cache locality, undefined behavior.
- Practice debugging race conditions and memory issues.
- Review your Samsung systems/firmware experience and identify 2 strong stories.
- Do one mock coding interview.

Day 4 — System Design I

- Design a large-scale authentication/OAuth platform.
- Cover authentication vs authorization, token lifecycle, key rotation, caching, availability, consistency and failure handling.
- Design a distributed notification/event system: queues, retries, idempotency, ordering, backpressure and DLQ.
- Prepare 2 architecture stories from Microsoft using Situation → Options → Decision → Trade-offs → Result.
- Practice one 60-minute system-design interview.

Day 5 — System Design II + Performance

- Design a high-performance C++ service.
- Focus on latency vs throughput, allocation, lock contention, thread pools, batching, zero-copy, profiling, p99 latency and CPU utilization.
- Review distributed systems: replication, sharding, caching, consistency, rate limiting.
- Practice explaining trade-offs at Principal level.
- Do one D.E. Shaw-style systems/design mock.

Day 6 — AI + Modern Engineering

- Design an AI/agent application: LLM → planner → tools → execution → memory → evaluation → safety.
- Review APIs, microservices, observability, containers/cloud fundamentals and event-driven architecture.
- Prepare a concise story around your agent/code-generation patent proposal and AI work.
- DSA: mixed timed set of 5 problems.
- Identify and close your top 5 technical gaps.

Day 7 — Principal Leadership

- Prepare 8–10 leadership stories: hardest problem, architecture, production incident, conflict, influence without authority, mentoring, disagreement, failure, performance improvement, initiative.
- For every story, emphasize scope, decisions, alternatives, trade-offs, influence and measurable outcome.
- Prepare answers for: Why leave Microsoft? Why D.E. Shaw? Why Google? Why Adobe?
- Practice your explanation of the WinUI/XAML specialization without sounding defensive.
- Complete one behavioral mock.

Day 8 — Google + Adobe Focus

- Google: DSA, distributed systems, scalability, ambiguity and leadership.
- Adobe: architecture, backend/systems fundamentals, collaboration, product thinking and leadership.
- Do 2 timed coding problems in the morning and one system-design problem in the afternoon.
- Review your resume line-by-line: every technology and project should be explainable deeply.
- Do a 45-minute mixed mock and record weak areas.

Day 9 — D.E. Shaw + C++ Intensive

- C++ deep dive: object model, templates, STL complexity, memory, concurrency and performance.
- Algorithms: graphs, DP, heaps, binary search and tricky data-structure problems.
- Systems: networking, latency, cache behavior, multithreading, lock contention and profiling.
- Practice a strong answer to: Why D.E. Shaw and why C++ now?
- Run a full 90-minute mock: coding + C++/systems + design.

Day 10 — Full Simulation + Light Review

- Morning: one coding interview under real conditions.
- Late morning: one C++/systems interview.
- Afternoon: one Principal-level system-design interview.
- Evening: behavioral round and company-specific questions.
- Do not learn new topics. Review only mistakes, cheat sheets and your strongest stories.
- Sleep properly and prepare interview logistics, environment and questions for the interviewer.

Daily Operating Rules

Area	Rule
Coding	2–3 focused hours; prioritize correctness, complexity and communication over problem count.
C++	1.5–2 hours; explain concepts aloud and write small examples when needed.
System Design	1.5–2 hours; always cover requirements, scale, architecture, data, failure modes, trade-offs and observability.
Leadership	45–60 minutes; rehearse stories, but do not memorize word-for-word.
Final review	30 minutes; maintain a running list of mistakes and revisit only those.

Interview Positioning

Target narrative: “I am a senior/principal-level software engineer with experience across systems, distributed components, authentication, large-scale Microsoft engineering and AI/agent work. My recent role has included Windows client technologies, but my core strength is solving complex engineering problems, designing systems, debugging production issues and leading technical decisions across teams. I now want to put systems and architecture more centrally into my role.”

What NOT to do

- Do not spend the 10 days learning a completely new language from scratch.
- Do not grind hundreds of LeetCode problems without reviewing mistakes.
- Do not apologize for WinUI/XAML; explain the transferable engineering depth behind the work.
- Do not memorize system-design diagrams. Learn the reasoning and trade-offs.
- Do not sacrifice sleep during the final 2–3 days.

Goal: You do not need to become a different engineer. You need to make your existing experience interview-ready, strengthen C++/systems fundamentals, and communicate your Principal-level impact clearly.